



DOCUMENTATION 4 : Provisioning Azure avec Terraform

Projet : CYNA

Auteur : Antoine Dissake DevOps

Date : Mai 2026

Objectif

Terraform automatise la création des **trois VMs Azure** du projet (Veeam, WSUS, SaaS) ainsi que le réseau virtuel associé. L'utilisation de l'Infrastructure as Code garantit que l'infrastructure peut être reconstruite à l'identique en quelques minutes, et que toute modification est tracée dans Git.

Prérequis

Le déploiement Terraform nécessite trois éléments. Un **abonnement Microsoft Azure** actif (fourni par les crédits étudiants). Un **Service Principal** créé avec la commande `az ad sp create-for-rbac` qui sert d'identité robot à Terraform pour s'authentifier auprès d'Azure. Et les outils **Terraform CLI** et **Azure CLI** installés sur la machine d'administration.

Structure du projet

Le code est organisé en plusieurs fichiers thématiques pour favoriser la lisibilité et la maintenance. Le fichier `providers.tf` déclare le provider Azure et sa version. Le fichier `variables.tf` centralise les variables paramétrables. Le fichier `network.tf` décrit le réseau Azure (VNet, subnets, NSG). Les fichiers `vm-veeam.tf`, `vm-wsus.tf` et `vm-saas.tf` décrivent chacun une VM avec ses spécificités. Enfin, le fichier `outputs.tf` expose les informations utiles après déploiement (IPs publiques notamment).

Ressources provisionnées

L'ensemble du déploiement Azure se compose d'un Resource Group conteneur, d'un Virtual Network en `172.16.0.0/16`, d'un subnet "serveurs" et d'un subnet "passerelle VPN", d'un Network Security Group filtrant le trafic entrant, des

trois VMs avec leurs cartes réseau et disques managés associés, et d'une passerelle VPN reliant le cloud aux sites Genève et Paris.

Workflow de déploiement

Le déploiement suit le cycle standard de Terraform en quatre commandes :

bash

```
terraform init      # Initialisation des providers
terraform validate  # Vérification de la syntaxe
terraform plan      # Prévisualisation des changements
terraform apply     # Création réelle des ressources
```

À l'issue, Terraform affiche les IPs publiques et les noms des VMs créées. La commande `terraform destroy` permet de supprimer proprement toutes les ressources en fin de cycle, ce qui est utile pour préserver les crédits Azure.

Gestion du state Terraform

Le **state file** (`terraform.tfstate`) contient l'état actuel de l'infrastructure et constitue un élément critique. Une perte ou une modification concurrente peut casser le déploiement. Pour sécuriser ce fichier, le projet utilise un **backend distant** stocké sur un Azure Blob Storage dédié, ce qui apporte trois avantages : le state est sauvegardé automatiquement et versionné, il est partagé entre les membres de l'équipe, et un mécanisme de verrouillage empêche que deux personnes lancent `terraform apply` simultanément.

Gestion des secrets

Les credentials Azure (Service Principal) ne sont jamais stockés dans le code Git. Ils sont définis comme **variables d'environnement** ou dans un fichier `terraform.tfvars` exclu du dépôt via le `.gitignore` . Cette discipline est cruciale pour éviter toute fuite de secrets dans l'historique Git.
